

1.程序是个状态机 理解 YEMU 的执行过程

YEMU 先把 16 和 33 加载致 R0 和 R1（但是 load 只支持加载到 R0，实际是先加载 33 到 R0 后再移动到 R1），然后计算 R0+R1,默认输出到 R0，然后把 R0 的值写入到内存，然后程序终止。YEMU 的执行方式为为，先根据 PC 取值并写入到 this.inst 里面，然后通过联合体，默认按照 R 类进行拆分来获取 OP，根据 OP 进行指令分类与译码、执行，然后更新 PC，接着根据 halt 判断是否继续，特别的是：一旦译码失败就直接停止执行

2.RTFSC 请整理一条指令在 NEMU 中的执行过程

指令在 main 进入 engine_start 之后开始执行，先在 cpu_exec 中的 execute 中的 exec_once 中，接收 pc 并计算 snpc，接着导入 isa_exec_once，在 isa_exec_once 里面进行

取指：使用 inst_fetch 从 snpc 中取出对应的指令,同时使得 snpc+=len，然后进入 decode_exec，进入正式的执行

译码：decode_exec 中通过 INSTPAT 里的 pattern_decode 比较关键位，判断指令类型，然后进入 INSTPAT_MATCH，根据匹配到的指令类型获取寄存器编号和不同类型的立即数

执行：执行 INSTPAT 判断确定的指令的任务，然后完成返回，一路退回到 exec_once

更新：通过 cpu.pc = s->dnpc;用动态新地址更新 pc

3.程序如何运行 理解打字小游戏如何运行

进入 main 之后，先调用 ioe_init()的 API 初始化 AM 方面的接口，然后进入 video_init(), 先获取窗口的大小，然后创建：blank 纯空白（和底色一样）的格子，用于盖掉字符移动后的位置；blank_line 空白行，用于初始化的时候将每一行的颜色都设为目标色（感觉其实直接用一个块也是可以的），不过用行可能是不用过度占用内存；然后 texture 字符的颜色集合，即 26 个字母，每个有 3 种状态，每个有 16*16 的大小的像素格子，根据 font.c 进行颜色填充，类似 spirit。初始化完成后进入主循环。

主循环里面有 3 块：首先是 FPS 的计算，然后在实现 FPS 的前提下更新物理引擎

(game_logic_update)：先是每个固定帧数就生成一个字符，指定它的位置，并随机赋予一个向下的速度（我通过插入了 speedDown 来调整速度，不然真的是太快了玩不过来）；然后是遍历每一个存在的字符，步进更新，如果是触底了给一个 30 帧的计时器，速度调成 0，30 帧后就或者是飞回到上边沿就消失。接下来进行按键处理，按 ESC 是退出，按有效按键进入 check_hit 进行检定，遍历所有存在的字符，找到存在的且是最下面的那个字符，如果成立就把字符速度反转并计分，否则记录 Wrong。然后进入 render 更新屏幕：先把所有字符的原本位置用 blank 给擦除，然后根据速度的大小（>0 是下落，=0 是触底 miss，<0 是 hit 往回飞）来指定颜色，从而调用 texture 写入对应的字符完成屏幕更新。

4.编译与链接 static inline inst_fetch()

static 的作用是使得 inst_fetch 的定义局限在每个引用了 ifetch.h 的文件里面，避免重复定义，而 inline 是在.c 内进行内联。实践证明，单独去掉都没问题，能通过 cpu-test，但是一起去掉就会发生 multiple definition of 'inst_fetch'，即重复定义，即两个都引用了 ifetch.h 的 inst_fetch，但是都可以作为外部定义，于是就导致了先处理了 hostcall.c，然后再处理 inst.c 的时候去翻译了其他文件（hostcall.c）中存在外部定义，就导致了重复定义报错。去掉 inline 不会报错是因为有 static 托底，保证函数的定义仅仅局限

于这两个文件中；而 inline，通过 RTFM WG14/N1256 ISO/IEC 9899:TC3 得知：“An inline definition does not provide an external definition for the function, and does not forbid an external definition in another translation unit.”即“内联定义不为该函数提供外部定义，也不禁止其他翻译单元中存在外部定义”，从而就不会报 multiple definition of 'inst_fetch'。

5.编译与链接 volatile static int dummy

我通过 `find . -name "*.o" | xargs nm 2>/dev/null | grep -E '[bB].*dummy$' | wc -l` 进行查找，先通过 find 找到所有的.o 文件，然后通过 nm 查找变量，并通过 grep 筛选出“b dummy”来计数，在 common.h 修改后的结果是 33，在 debug.h 修改后的结果也是 33（即使是改成了 dummy1 的结果也是一样的 33），两者重合度极高。后根据 RTFSC，发现 debug.h 的唯一引用就是在 common.h 里面，进而导致了两者的完美重合，任何一个.c 只要引用了 common.h 就相当于引用了 debug.h，从而导致输出结果是一样的。以及，common.h 又被包含在了 debug.h 里面。所以，在定义了这两个变量之后，进而就导致了循环爆出重复定义的问题。根据 RTFM WG14/N1256 ISO/IEC 9899:TC3 得知：

即在有多个预定义的情况下是可以共存的，但是如果有一个进行了赋值定义，那么就相当于是所有的 dummy 的 tentative definition 失效，需要进行重定义检查，进而就发生了上文提及的循环定义报错。

A declaration of an identifier for an object that has file scope without an initializer, and without a storage-class specifier or with the storage-class specifier **static**, constitutes a *tentative definition*. If a translation unit contains one or more tentative definitions for an identifier, and the translation unit contains no external definition for that identifier, then the behavior is exactly as if the translation unit contains a file scope declaration of that identifier, with the composite type as of the end of the translation unit, with an initializer equal to 0.

If the declaration of an identifier for an object is a tentative definition and has internal linkage, the declared type shall not be an incomplete type.

```
In file included from /home/biruide/ysyx-workbench/nemu/include/utils.h:19,
from /home/biruide/ysyx-workbench/nemu/include/debug.h:21,
from /home/biruide/ysyx-workbench/nemu/include/common.h:47,
from src/device/device.c:16:
/home/biruide/ysyx-workbench/nemu/include/common.h:53:21: error: redefinition of 'dummy'
53 | volatile static int dummy = 0;
    | ^~~~~~
In file included from /home/biruide/ysyx-workbench/nemu/include/debug.h:19,
from /home/biruide/ysyx-workbench/nemu/include/common.h:47,
from src/device/device.c:16:
/home/biruide/ysyx-workbench/nemu/include/common.h:53:21: note: previous definition of 'dummy' with type 'int'
53 | volatile static int dummy = 0;
    | ^~~~~~
In file included from /home/biruide/ysyx-workbench/nemu/include/common.h:47,
from src/device/device.c:16:
/home/biruide/ysyx-workbench/nemu/include/debug.h:45:21: error: redefinition of 'dummy'
45 | volatile static int dummy = 0;
    | ^~~~~~
In file included from /home/biruide/ysyx-workbench/nemu/include/utils.h:19,
from /home/biruide/ysyx-workbench/nemu/include/debug.h:21,
from /home/biruide/ysyx-workbench/nemu/include/common.h:47,
from src/device/device.c:16:
/home/biruide/ysyx-workbench/nemu/include/common.h:53:21: note: previous definition of 'dummy' with type 'int'
53 | volatile static int dummy = 0;
    | ^~~~~~
In file included from src/device/device.c:16:
/home/biruide/ysyx-workbench/nemu/include/common.h:53:21: error: redefinition of 'dummy'
53 | volatile static int dummy = 0;
    | ^~~~~~
In file included from /home/biruide/ysyx-workbench/nemu/include/common.h:47,
from src/device/device.c:16:
/home/biruide/ysyx-workbench/nemu/include/debug.h:45:21: note: previous definition of 'dummy' with type 'int'
45 | volatile static int dummy = 0;
    | ^~~~~~
make: *** [/home/biruide/ysyx-workbench/nemu/scripts/build.mk:34: /home/biruide/ysyx-workbench/nemu/build/obj-riscv32-nemu-interpret/src/device/device.o] Error 1
```

6.了解 Makefile

键入 `make ARCH=riscv32-nemu` 之后，由于没有指定操作，默认目标为 image，由 hello/Makefile 进入 abstract-machine/Makefile，并根据 ARCH 进入 riscv32-nemu.mk，nemu.mk 和 riscv.mk，从中读出 image 的依赖 image-dep 返回 a-m 的 Makefile，获取目标\$(IMAGE).elf 其中的 IMAGE=绝对路径/hello/build/hello-riscv32-nemu，然后其又依赖 LINKAGE，即 build/riscv32-nemu/下面的所有来自 src 的.c=>.o 文件，即 hello.o，从而运行\$(DST_DIR)/%.o: %.c，其中的@\$(CC) -std=gnu11 \$(CFLAGS) -c -o \$@ \$(realpath \$<)的参数分别为：CC=\$(CROSS_COMPILE)gcc，其中 CROSS_COMPILE=riscv64-linux-gnu-来自 riscv.mk，CFLAGS 来自以上的三个 AM 文件，而\$@ \$(realpath \$<)分别引用 hello.o 和 hello.c，即使用参数将 hello.c 编译为 hello.o。此外，由于 LINKAGE 还被添加了一个(ARCH).a，在 am 下调用 make archive，编译 src/platform 和 klib/src 下的所有的.c 文件为.o，并使用 AR 生成 klib-riscv32-nemu.a，然后返回\$(IMAGE).elf，使用\$(LD) \$(LDFLAGS) -o \$@ --start-group \$(LINKAGE) --end-group 将它们进行链接，其中 LDFLAGS 来自 nemu.mk，生成 hello-riscv32-nemu.elf，最后再通过反汇编生成 hello-riscv32-nemu.txt 和把.elf 转换为.bin，等待调用或者结束。